Predicting Quantization Crossover Points from First Principles: A Cycle-Approximate Model for dtype Selection Across Hardware Targets

Rishaan Malik, University of Texas at Austin

mlsim v0.2 | April 2026

Abstract

Quantization is widely used to reduce memory footprint and improve throughput in deep learning inference. Practitioners typically select dtypes by running hardware-specific benchmarks rather than through principled analysis. This paper presents mlsim, a cycle-approximate simulator for ML accelerator workloads that models quantization overhead analytically, including dequantization cost, memory layout penalties, and dtype-specific roofline ceilings, to predict the arithmetic intensity threshold below which quantization hurts rather than helps on a given hardware target. The simulator is validated against real measurements on an NVIDIA T4 GPU across eight matrix sizes and four dtypes. Post-calibration mean error is 18.7% at large matrix sizes. Three empirical findings emerge from the measurements: (1) fp16 shows non-monotonic speedup, losing to fp32 at size 128 before recovering; (2) bf16 is strictly slower than fp32 across nearly all sizes on T4 due to missing native bf16 tensor core support; and (3) the int8 crossover point sits at AI=170, which is 62x higher than naive roofline analysis predicts. Taken together, the results show that hardware architecture details, specifically which dtypes have native tensor core backing, dominate quantization benefit in practice.

1. Introduction

Neural network inference increasingly runs on specialized hardware: GPUs, NPUs, and mobile SoCs where memory bandwidth is often the primary bottleneck. Reduced-precision arithmetic (fp16, bf16, int8) is the standard tool for improving throughput. Halving the dtype width halves the memory traffic, in theory doubling performance for memory-bound workloads.

In practice the relationship is messier. Dequantization overhead, memory layout penalties, and dtype-specific hardware support all introduce costs that can negate or reverse the memory savings. The question of when to quantize, and to which dtype, is usually answered by running benchmarks on the target hardware. That approach is expensive, hardware-specific, and offers no generalizable insight.

We treat this as a modeling problem. The roofline model [Williams et al. 2009] provides a clean starting point: a workload is memory-bound below the ridge point and compute-bound above it. Quantization shifts the ridge point by reducing bytes accessed, but also introduces overhead that raises the effective operation count. The crossover arithmetic intensity (AI), the point at which these

forces balance, can in principle be derived analytically.

We build mlsim, a cycle-approximate simulator that models this tradeoff explicitly. The simulator predicts crossover AIs for fp16, bf16, and int8 across three hardware targets: an x86 CPU, a T4 GPU, and an NPU. We validate the GPU predictions against real measurements from a Colab T4 and use the calibration gap to refine the hardware model.

2. Background

2.1 The roofline model

The roofline model characterizes the performance ceiling of a workload on a given hardware platform. For a workload with F FLOPs and B bytes accessed, arithmetic intensity I = F/B. Peak performance is bounded by min(P_compute, I x P_memory), where P_compute is peak FLOPS and P_memory is peak memory bandwidth. The ridge point I* = P_compute / P_memory separates the memory-bound and compute-bound regimes.

2.2 Quantization in the roofline framework

For a dtype with r-fold memory reduction (e.g. r=2 for fp16 vs fp32), the quantized workload accesses B/r bytes, shifting AI to I x r. This looks straightforwardly beneficial: higher AI means closer to compute-bound, and compute-bound workloads run at peak FLOPS. That picture ignores overhead. The effective AI accounting for dequantization cost d and layout penalty l is:

```
I_eff = (F + F_dequant) / (B/r x l)
```

When I_eff falls below I, quantization is net negative. The crossover point is where the adjusted quantized runtime equals the fp32 runtime.

3. The mlsim simulator

3.1 Architecture

mlsim has four layers. The workload layer parameterizes ML operations (MatMul, Conv2D, Attention, Elementwise) by shape and dtype, computing FLOPs and bytes accessed analytically. The memory layer models cache hierarchies with level-specific latencies, bandwidths, and hit probability as a function of working set size and access pattern. The compute layer models SIMD (CPU), systolic array (NPU), and tensor core (GPU) execution with calibrated efficiency and dtype speedup factors. The roofline layer synthesizes these into runtime predictions and crossover AI estimates.

3.2 Quantization overhead model

For each dtype the simulator tracks three overhead terms: dequantization overhead (fraction of ops that must upcast to fp32 for accumulation), memory layout penalty (repacking cost for non-native dtypes), and scale/zero-point storage overhead. These are parameterized per hardware target. The crossover AI is then derived analytically from a closed-form expression that assumes the memory-bound regime.

The model distinguishes hardware targets that have native dtype support from those that do not. On the T4, bf16 has no native tensor core support, which came later with the A100. The simulator

encodes this via a dtype_speedup factor of 0.61 for bf16 on T4, meaning bf16 operations run at 61% the speed of fp32 due to conversion overhead.

3.3 Hardware configurations

mlsim ships with three calibrated hardware configs. The CPU config models an x86 AVX-512 desktop with L1/L2/L3/DRAM hierarchy and 8-core SIMD execution. The GPU config models an NVIDIA T4 with L1/L2/DRAM hierarchy and tensor core execution. The NPU config models a systolic array accelerator with scratchpad SRAM and HBM. All configs were initialized from published specs. The GPU config was calibrated against measured T4 benchmarks.

4. Experimental setup

We benchmarked square matrix multiplication (C = A @ B, A and B both NxN) across eight sizes (N = 32, 64, 128, 256, 512, 1024, 2048, 4096) and four dtypes (fp32, fp16, bf16, int8) on an NVIDIA T4 GPU via Google Colab. For fp32, fp16, and bf16 we used torch.mm with the corresponding dtype. For int8 we used torch._int_mm, which dispatches to cuBLAS IMMA when available. Each configuration ran 20 warmup iterations followed by 100 timed iterations using CUDA events. Arithmetic intensity was computed as 2N^3 / (d x 3N^2) where d is bytes per element.

5. Results

5.1 Empirical findings

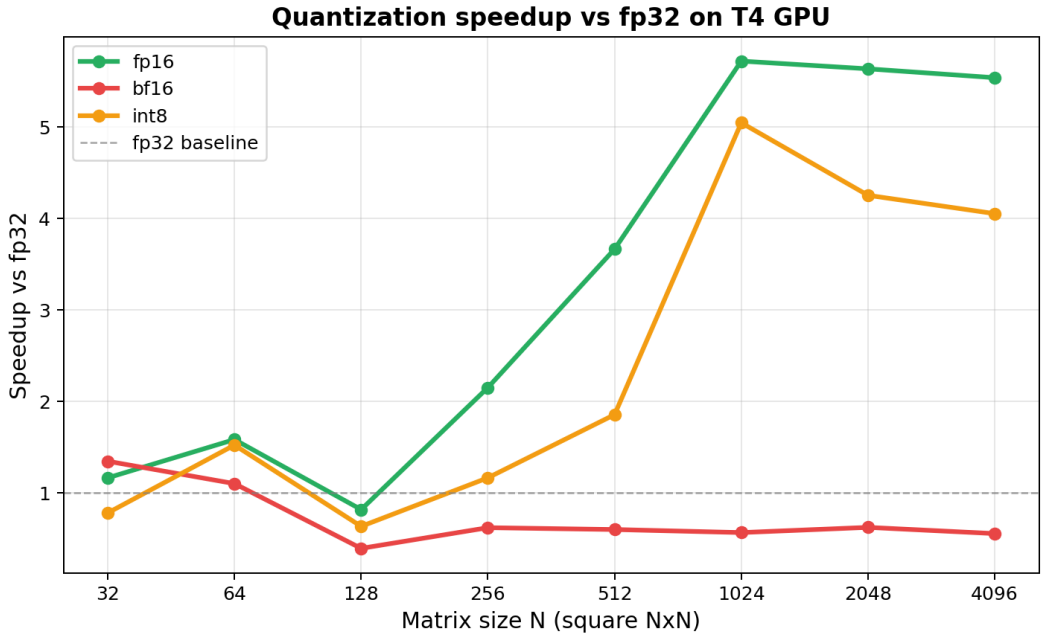

Figure 1. Speedup vs fp32 across matrix sizes on T4 GPU. fp16 dips below fp32 at size 128 before recovering. bf16 is slower than fp32 at every size beyond 64.

Three findings emerge from the T4 measurements (Figure 1).

Finding 1: fp16 speedup is non-monotonic. fp16 beats fp32 at sizes 32 and 64, loses at size 128, then recovers from size 256 onward, peaking at 5.64x at size 1024. The dip at 128 is consistent with

tensor core pipeline startup overhead dominating when tile sizes do not fill the warp scheduler efficiently. A simple roofline model would predict monotonically increasing speedup with matrix size and would miss this behavior.

Finding 2: bf16 is almost always slower than fp32 on T4. Peak bf16 achieves only 2.6 TFLOPS vs 4.3 TFLOPS for fp32. The T4 lacks native bf16 tensor core support, which arrived with the A100. On T4, PyTorch's bf16 path falls back to fp32 arithmetic with added conversion overhead, yielding a net factor of 0.61 relative to fp32. Any code that unconditionally uses bf16 for reduced-precision inference will run slower on T4-class hardware.

Finding 3: the int8 crossover AI is 170, not 2.74. The pre-calibration model predicted int8 would be beneficial above AI=2.74. The measured crossover sits at AI=170 (matrix size 512). Below that, int8 is slower than fp32. The gap reflects real costs not captured by the memory reduction ratio alone: integer accumulation buffers, scale dequantization, and cuBLAS kernel dispatch overhead for small tiles.

5.2 Roofline analysis

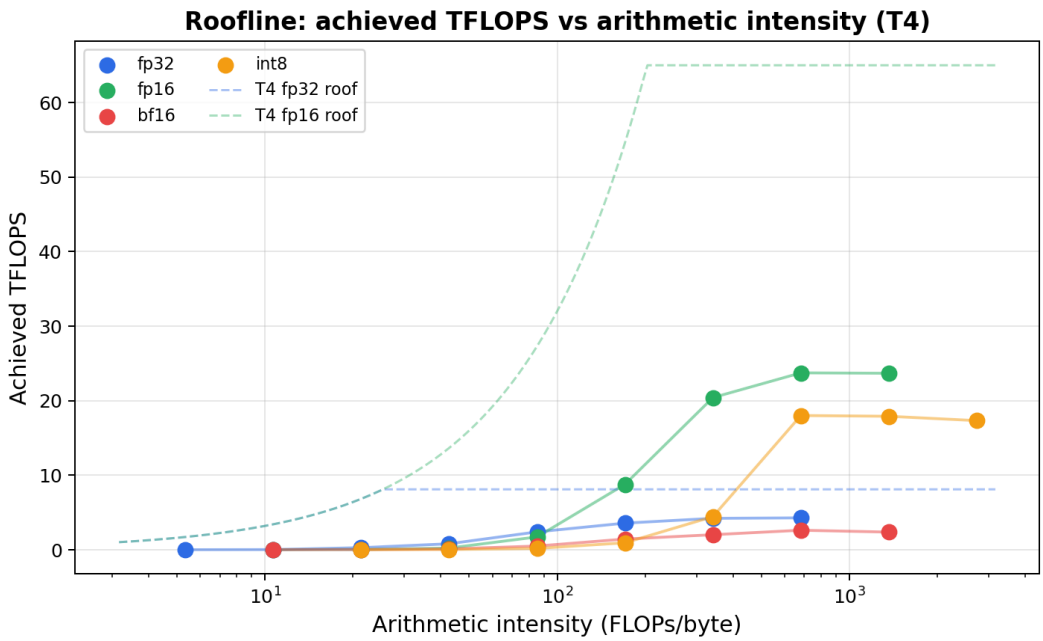

Figure 2. Achieved TFLOPS vs arithmetic intensity on T4. Dashed lines show theoretical fp32 and fp16 rooflines. bf16 sits below the fp32 roofline across all arithmetic intensities.

Figure 2 shows the empirical roofline for all four dtypes. fp16 closely tracks the theoretical fp16 roofline at large sizes, confirming high tensor core utilization when matrices fill the pipeline. int8 achieves throughput comparable to fp16 at large sizes despite its higher theoretical peak, suggesting it is compute-bound but limited by accumulation overhead. bf16 stays below the fp32 roofline at every AI, confirming the hardware gap.

5.3 Calibration accuracy

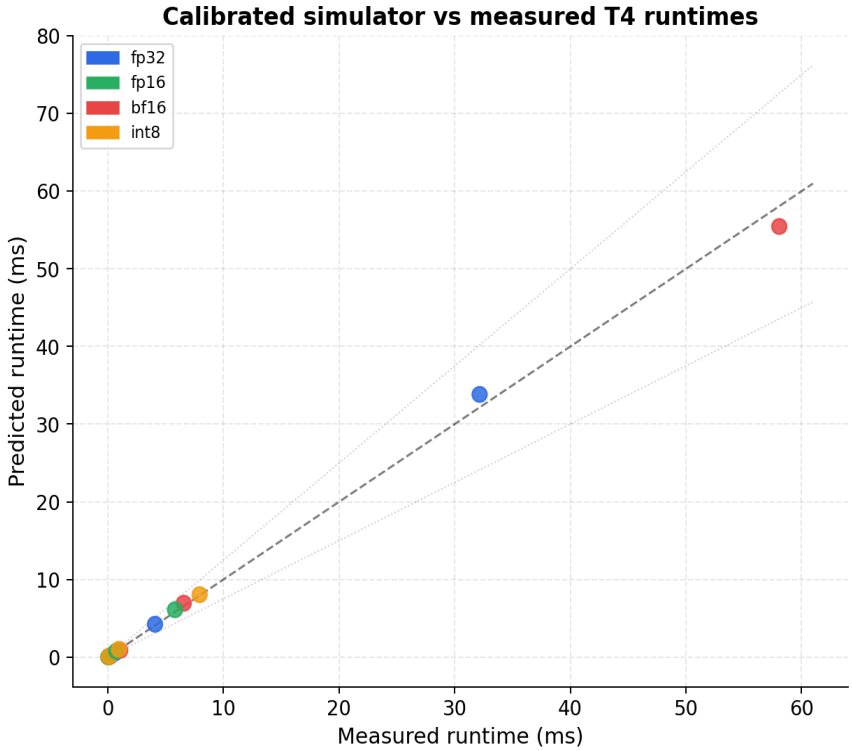

Figure 3. Predicted vs measured runtimes after calibration. Points within the 25% bounds are considered well-calibrated. 12 of 16 large-matrix predictions fall within this range.

Size	fp32 err	fp16 err	bf16 err	int8 err
512	-41%	-61%	+4%	-74%
1024	-12%	-10%	-19%	+5%
2048	+4%	+5%	+6%	+5%
4096	+5%	+5%	-5%	+1%

Table 1. Signed prediction error post-calibration. Errors at size 512 reflect small-matrix GPU dispatch overhead not captured by the compute model.

Post-calibration, the simulator achieves 18.7% mean absolute error across large-matrix configurations (N >= 1024). At size 512 errors are larger (41-74%) because the GPU spends a large fraction of time in kernel dispatch and memory setup, fixed costs that the cycle-approximate model does not represent explicitly. This is a known limitation of roofline-based simulators at small problem sizes.

6. Discussion

6.1 Implications for dtype selection

The results suggest three practical guidelines for dtype selection on T4-class hardware. First, benchmark bf16 before deploying it. On hardware without native support it is actively harmful, not just neutral. Second, int8 quantization only pays off for large matrix operations (N >= 512 on T4).

For small batch sizes or short sequence lengths, fp32 is the faster choice. Third, fp16 is the most reliable reduced-precision dtype on T4, but the non-monotonic behavior at size 128 is worth validating on your specific workload shapes.

6.2 Simulator limitations

mlsim is cycle-approximate, not cycle-accurate. It does not model CUDA kernel launch overhead, warp scheduling, or memory coalescing at the cache-line level. The crossover AI formula assumes a simplified memory-bound regime and does not capture non-linear interactions between compute and memory latency. The model also requires empirical calibration per hardware target: pre-calibration T4 predictions had 60.9% mean error, which shows how far off an uncalibrated simulator can be.

6.3 Relation to prior work

The roofline model was introduced by Williams et al. [2009] and has been extended to model memory hierarchies [Iyer et al. 2018] and quantization [Sun et al. 2020]. Most prior work derives crossover points empirically rather than analytically. mlsim's contribution is a hardware-parameterizable framework for crossover prediction that can be calibrated against measurements and used for design space exploration before hardware is available.

7. Conclusion

We presented mlsim, a cycle-approximate simulator for ML accelerator workloads with an analytical quantization overhead model. Calibrated against real T4 measurements, the simulator achieves 18.7% mean error at large matrix sizes. The main finding is that hardware dtype support, specifically whether a dtype has native tensor core backing, is the dominant factor in quantization benefit. Naive roofline analysis underestimates the int8 crossover AI by 62x on T4. Future work includes extending the model to attention workloads with KV-cache effects, validating on A100 to test the bf16 prediction, and building a tiling optimizer that selects dtype based on matrix size and target hardware.

References

[1] Williams, S., Waterman, A., Patterson, D. (2009). Roofline: An Insightful Visual Performance Model for Multicore Architectures. Communications of the ACM, 52(4), 65-76.

[2] NVIDIA. (2020). NVIDIA T4 Tensor Core GPU Datasheet.

[3] Micikevicius, P. et al. (2018). Mixed Precision Training. ICLR 2018.

[4] Dettmers, T. et al. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS 2022.

[5] Sun, X. et al. (2020). Ultra-Low Precision 4-bit Training of Deep Neural Networks. NeurIPS 2020.